

Data Wrangling en R 2

2026-03-26

Tabla de contenidos

De qué trata la clase de hoy	1
3. Tercera Parte	2
Librerías y los datos	2
Normalización de variables y remoción de datos faltantes	4
4. Correr scripts en segundo plano	14

De qué trata la clase de hoy

Hoy vamos a continuar con los temas de la clase pasada, pero vamos a utilizar otras herramientas que nos van a facilitar muchas de las tareas que veníamos haciendo.

1. **Primera parte:** vamos a ver las consignas del TP.
2. **Segunda parte:** vamos a revisar el final de la clase anterior, que quedó inconcluso y tenía algunos bugs.
3. **Tercera parte:** vamos a ver otra base de datos. La idea en este caso es volver a hacer algunas de las operaciones que hicimos antes valiéndonos de `janitor` y `stringi`, que son librerías que sintetizan operaciones de limpieza que resultan engorrosas de escribir a mano. La idea es que este *notebook* y el anterior les sirva de machete para que puedan hacer más tranquilos/as el TP individual. Vamos a ver, además, cómo ordenar variables en forma jerárquica.
4. **Cuarta parte:** seguramente la clase que viene veremos cómo organizar *scripts* dentro de un proyecto grande para que cada archivo realice una tarea en particular.

3. Tercera Parte

Vamos a trabajar con datos del **SINTA (Sistema de Inteligencia Turística Argentina)**, que es el portal de datos abiertos de la Subsecretaría de Turismo. Utilizaremos específicamente dos bases de datos de la **Encuesta de Turismo Internacional (ETI)**: una correspondiente al **Aeropuerto Internacional de Córdoba** y otra que agrupa el movimiento de **CABA y PBA** a través de los arribos en **Ezeiza y Aeroparque**.

Estas bases nos permiten analizar el volumen de **turistas no residentes**, sus **pernoctes** y la **estadía media** según su país de residencia, lo que nos servirá de insumo para practicar técnicas de limpieza, normalización y transformación de datos.

En este caso no vamos a trabajar sobre una hipótesis particular porque armé el análisis para que se puedan realizar las operaciones que nos faltan revisar.

Librerías y los datos

```
# install.packages(c("tidyverse", "janitor"))
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.1.4      v readr      2.1.6
v forcats    1.0.1      v stringr    1.6.0
v ggplot2    4.0.1      v tibble     3.3.0
v lubridate  1.9.4      v tidyr      1.3.2
v purrr      1.2.0
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
library(janitor)
```

Attaching package: 'janitor'

The following objects are masked from 'package:stats':

chisq.test, fisher.test

```
library(stringi)
```

```
path_ba = "../data/estadias_pba_caba.csv"
estadias_ba = read_csv(
  path_ba,
  # skip=0 si tiene comentarios en las primeras antes de la tabla, este parámetro indica la
)
```

Rows: 432 Columns: 6

-- Column specification -----

Delimiter: ","

chr (2): país de residencia, observaciones

dbl (3): turistas_no_residentes, pernoctes_no_residentes, estadia_media_no_...

date (1): indice_tiempo

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

```
head(estadias_ba)
```

A tibble: 6 x 6

	indice_tiempo	país de residencia	turistas_no_residentes
	<date>	<chr>	<dbl>
1	2014-01-01	Bolivia	17181
2	2014-01-01	Brasil	149582
3	2014-01-01	Chile	48953
4	2014-01-01	Paraguay	17316
5	2014-01-01	Uruguay	11429
6	2014-01-01	EE.UU y Canadá	82090

i 3 more variables: pernoctes_no_residentes <dbl>,

estadia_media_no_residentes <dbl>, observaciones <chr>

```
path_co = "../data/estadias_cordoba.csv"
estadias_co = read_csv(path_co)
```

Rows: 240 Columns: 6

-- Column specification -----

Delimiter: ","

chr (2): país de residencia, observaciones

dbl (3): turistas_no_residentes, pernoctes_no_residentes, estadia_media_no_...

```
date (1): indice_tiempo
```

i Use ``spec()`` to retrieve the full column specification for this data.

i Specify the column types or set ``show_col_types = FALSE`` to quiet this message.

```
head(estadias_co)
```

```
# A tibble: 6 x 6
  indice_tiempo `país de residencia`   turistas_no_residentes
  <date>        <chr>                  <dbl>
1 2014-01-01    Brasil                  1964
2 2014-01-01    Chile                  3138
3 2014-01-01    EE.UU, Canadá y México  3635
4 2014-01-01    Resto de América        2290
5 2014-01-01    Europa y resto del mundo  2159
6 2014-04-01    Brasil                  2049
# i 3 more variables: pernoctes_no_residentes <dbl>,
#   estadia_media_no_residentes <dbl>, observaciones <chr>
```

Normalización de variables y remoción de datos faltantes

Armamos una función que, dado un dataframe, realice ambas operaciones

```
normalize_dataframe = function(dataframe, clean = TRUE, which_remove = "cols") {
  # Normalizacion
  if (clean) {
    data_clean = clean_names(dataframe)
  } else {
    data_clean = dataframe
  }

  # Eliminacion de datos faltantes
  if (which_remove == "both") {
    data_clean = data_clean |>
      janitor::remove_empty(which = 'cols') |>
      janitor::remove_empty(which = 'rows')
  } else {
    data_clean = janitor::remove_empty(data_clean, which = which_remove)
  }

  return(data_clean)
}
```

```
}
# normalize_dataframe(estadias_co, clean = FALSE, which_remove = 'both')
```

Esta función recibe como input un dataframe, normaliza los nombres de las variables (columnas) y, luego, remueve las observaciones (rows) o variables (cols) que tengan todos sus valores faltantes (NA).

Para llamar una función que está definida en dos librerías diferentes con el mismo nombre, podemos escribir primero la librería creadora para desambiguar el conflicto —`janitor::clean_names()`. En este ejemplo especificar la librería padre está de más porque `clean_names` no se define en otras librerías.

En caso de querer desambiguar una función *built-in* (pre definida en R), podemos hacerlo así: `base::una_funcion()`.

```
estadias_co_clean = normalize_dataframe(estadias_co, which_remove = 'both')
estadias_ba_clean = normalize_dataframe(estadias_ba, which_remove = "both")
cat(
  c('Estadías Córdoba antes:', nrow(estadias_co), 'y después:', nrow(estadias_co_clean))
)
```

Estadías Córdoba antes: 240 y después: 239

```
cat(
  c('\nEstadías CABA antes:', nrow(estadias_ba), 'y después:', nrow(estadias_ba_clean))
)
```

Estadías CABA antes: 432 y después: 430

Notemos que hay países que no comparten las provincias, entonces es conveniente hacer un `inner_join`.

```
estadias_ba_clean |> select(pais_de_residencia) |> unique()
```

```
# A tibble: 9 x 1
  pais_de_residencia
  <chr>
1 Bolivia
2 Brasil
```

- 3 Chile
- 4 Paraguay
- 5 Uruguay
- 6 EE.UU y Canadá
- 7 Resto América
- 8 Europa
- 9 Resto del Mundo

```
estadias_co_clean |> select(pais_de_residencia) |> unique()
```

```
# A tibble: 5 x 1
  pais_de_residencia
  <chr>
1 Brasil
2 Chile
3 EE.UU, Canadá y México
4 Resto de América
5 Europa y resto del mundo
```

Pero hay que tener cuidado ya que hay países que significan lo mismo, pero tienen nombres diferentes. Por ejemplo, “Resto de América” y “Resto América”. En este caso, podemos revisar las categorías a mano dado que son pocas, de todos modos hagamos un tratamiento general.

```
estadias_ba_clean = estadias_ba_clean |>
  mutate(
    pais_de_residencia = pais_de_residencia |>
      str_to_lower() |> # todo a minúsculas
      stri_trans_general("Latin-ASCII") |> # quita tildes
      str_replace_all(" ", "_") |> # espacios por guiones
      str_remove_all("[[:punct:]]") |> # quita puntos y signos restantes
      str_trim() # elimina espacios no detectados
  ) |>
  mutate(
    pais_de_residencia = case_when(
      str_starts(pais_de_residencia, "eeuu") |
      str_starts(pais_de_residencia, "euro") |
      str_starts(pais_de_residencia, "resto") ~ "nolimitrofe", # la tilde se usa para separar
      .default = pais_de_residencia # mantiene el nombre original si no es 'otros'
    )
  )
estadias_co_clean = estadias_co_clean |>
```

```

mutate(
  pais_de_residencia = pais_de_residencia |>
    str_to_lower() |> # todo a minúsculas
    stri_trans_general("Latin-ASCII") |> # quita tildes
    str_replace_all(" ", "_") |> # espacios por guiones
    str_remove_all("[[:punct:]]") |> # quita puntos y signos restantes
    str_trim() # elimina espacios no detectados
) |>
mutate(
  pais_de_residencia = case_when(
    str_starts(pais_de_residencia, "eeuu") |
    str_starts(pais_de_residencia, "euro") |
    str_starts(pais_de_residencia, "resto") ~ "nolimitrofe",
    .default = pais_de_residencia # mantiene el nombre original si no es 'otros'
  )
)
estadias_ba_clean |> select(pais_de_residencia) |> unique()

```

```

# A tibble: 6 x 1
  pais_de_residencia
  <chr>
1 bolivia
2 brasil
3 chile
4 paraguay
5 uruguay
6 nolimitrofe

```

```

estadias_co_clean |> select(pais_de_residencia) |> unique()

```

```

# A tibble: 3 x 1
  pais_de_residencia
  <chr>
1 brasil
2 chile
3 nolimitrofe

```

Ahora sí, notamos que Córdoba registra ingresos por avión de brasileiros, chilenos y países no limítrofes, mientras que BA tiene ingresos más diversos.

Atención: esto no significa que Córdoba no tenga ingresos de otros países, puede que vuelos no directos no se registren de la misma forma (y por eso, no aparezcan en la tabla), o que sea conveniente aterrizar en BA e ir en otro medio de transporte (micro por ej.)

Otro detalle a tener en cuenta es que como agrupamos categorías en `pais_de_residencia` ahora habrá índices de tiempo repetidos para un mismo país, lo cual puede conllevar a errores más adelante.

EEUU, Mexico 2022/01/01 -> nolimitrofe 2022/01/01

Mexico 2022/01/01 -> nolimitrofe 2022/01/01

```
estadias_ba_clean = estadias_ba_clean |>
  group_by(pais_de_residencia, indice_tiempo) |> # agrupamos
  summarise( # tomamos valores medios /sumas según el indicador
    turistas_no_residentes = sum(turistas_no_residentes, na.rm = TRUE),
    pernoctes_no_residentes = sum(pernoctes_no_residentes, na.rm = TRUE),
    estadia_media_no_residentes = mean(estadia_media_no_residentes, na.rm = TRUE),
    .groups = "drop" # Para que el DF deje de estar agrupado tras el cálculo
  )
estadias_co_clean = estadias_co_clean |>
  group_by(pais_de_residencia, indice_tiempo) |> # agrupamos
  summarise( # tomamos valores medios /sumas según el indicador
    turistas_no_residentes = sum(turistas_no_residentes, na.rm = TRUE),
    pernoctes_no_residentes = sum(pernoctes_no_residentes, na.rm = TRUE),
    estadia_media_no_residentes = mean(estadia_media_no_residentes, na.rm = TRUE),
    .groups = "drop" # Para que el DF deje de estar agrupado tras el cálculo
  )
estadias_co_clean |> filter(pais_de_residencia=='nolimitrofe')
```

A tibble: 48 x 5

	pais_de_residencia	indice_tiempo	turistas_no_residentes
	<chr>	<date>	<dbl>
1	nolimitrofe	2014-01-01	8084
2	nolimitrofe	2014-04-01	4163
3	nolimitrofe	2014-07-01	7086
4	nolimitrofe	2014-10-01	6325
5	nolimitrofe	2015-01-01	6239
6	nolimitrofe	2015-04-01	5812
7	nolimitrofe	2015-07-01	6798
8	nolimitrofe	2015-10-01	7808
9	nolimitrofe	2016-01-01	8374
10	nolimitrofe	2016-04-01	5766


```
# i 38 more rows
# i 2 more variables: pernoctes_no_residentes <dbl>,
#   estadia_media_no_residentes <dbl>
```

Ahora sí, teniendo todo ordenado podemos unir las dos tablas y ver, como anticipábamos, que sólo tienen en común Brasil, Chile y no limítrofes.

```
estadias_totales = estadias_ba_clean |>
  inner_join(
    estadias_co_clean,
    by=c("pais_de_residencia","indice_tiempo"),
    suffix = c("_BA", "_CO") # el sufijo asignado a las tablas izq y derecha (reemplaza el v
  ) |>
  select(!starts_with('obs'))

estadias_totales |> head(15)
```

```
# A tibble: 15 x 8
  pais_de_residencia indice_tiempo turistas_no_residentes_BA
  <chr>               <date>               <dbl>
1 brasil             2014-01-01             149582
2 brasil             2014-04-01             175018
3 brasil             2014-07-01             224259
4 brasil             2014-10-01             180361
5 brasil             2015-01-01             145922
6 brasil             2015-04-01             147540
7 brasil             2015-07-01             182629
8 brasil             2015-10-01             116157
9 brasil             2016-01-01              83492
10 brasil            2016-04-01             127142
11 brasil            2016-07-01             183799
12 brasil            2016-10-01             145472
13 brasil            2017-01-01             108533
14 brasil            2017-04-01             146959
15 brasil            2017-07-01             204501
# i 5 more variables: pernoctes_no_residentes_BA <dbl>,
#   estadia_media_no_residentes_BA <dbl>, turistas_no_residentes_CO <dbl>,
#   pernoctes_no_residentes_CO <dbl>, estadia_media_no_residentes_CO <dbl>
```

```
print(estadias_totales |> select(pais_de_residencia) |> unique())
```

```
# A tibble: 3 x 1
  pais_de_residencia
  <chr>
1 brasil
2 chile
3 nolimitrofe
```

Si observamos el `indice_tiempo` las fechas parecieran repetirse cada 3 meses, esto es porque los datos se actualizan trimestralmente, lo cual podríamos haber anticipado si leíamos la página de donde descargamos los datos.

```
estadias_totales_cuatrimestrales = estadias_totales |>
  mutate(
    anio=year(indice_tiempo),
    trimestre=case_when(
      month(indice_tiempo) == 1 ~ 1,
      month(indice_tiempo) == 4 ~ 2,
      month(indice_tiempo) == 7 ~ 3,
      month(indice_tiempo) == 10 ~ 4,
      .default = NA # Opcional: para manejar meses que no coincidan
    )
  ) |> select(!indice_tiempo)

# Versión tidy
estadias_totales_cuatrimestrales = estadias_totales |>
  mutate(
    anio=year(indice_tiempo),
    trimestre=quarter(indice_tiempo)
  ) |> select(!indice_tiempo)
```

El objetivo de este bloque final es integrar herramientas de programación, pensando en cómo sintetizar operaciones en un *script*.

- **Automatización mediante loops:** el uso de un bucle `for` permite escalar el análisis de forma eficiente. En lugar de repetir el código para cada mercado, generamos reportes estandarizados para Brasil, Chile y países No limítrofes con un solo bloque de ejecución.
- **Jerarquía y factores:** la conversión de la columna `trimestre` a **factor** (`fct`) no es estética; es técnica. Garantiza que el orden cronológico (1 al 4) se preserve en cualquier análisis o gráfico posterior, evitando el ordenamiento alfabético por defecto.
- **Persistencia de datos (.csv vs .rds):** se guardan los datos en dos formatos distintos. El `.csv` permite la interoperabilidad con otras herramientas (como Excel), mientras que

el `.rds` es muy útil en R porque es el único formato que guarda los **metadatos**, como los niveles de los factores que acabamos de configurar.

```
jerarquia = c(
  "trimestre",
  "porcentaje_CO",
  "turistas_no_residentes_BA",
  "turistas_no_residentes_CO"
)
lista_bases = list(
  "Brasil" = 'brasil',
  "Chile" = 'chile',
  "No limítrofes" = 'nolimitrofe'
)
for (nombre in names(lista_bases)) {
  # Nos quedamos con una tabla para cada categoría
  codigo_pais_residente = lista_bases[[nombre]]

  resumen = estadias_totales_cuatrimestrales |>
    filter(
      pais_de_residencia == codigo_pais_residente,
      anio>=2015,
      2025>=anio
    ) |>
    select(!c(pais_de_residencia, anio)) |>
    group_by(
      trimestre
    ) |>
    summarize(
      turistas_no_residentes_BA = sum(
        turistas_no_residentes_BA, na.rm=TRUE
      ),
      turistas_no_residentes_CO = sum(
        turistas_no_residentes_CO, na.rm=TRUE
      ),
      .groups = "drop"
    ) |>
    mutate(
      porcentaje_CO = 100*turistas_no_residentes_CO/(turistas_no_residentes_BA +turistas_no_r
    )

  # Ordenamos las columnas jerárquicamente las columnas (esto NO es un factor)
  resumen = resumen |> select(all_of(jerarquia))
}
```

```

# Convertimos 'trimestre' en factor para asegurar orden cronológico (esto SÍ)
resumen$trimestre = factor(resumen$trimestre, levels = 1:4)

print(
  paste("Procesando:", nombre)
)
print(resumen)

# Definimos la ruta de salida
dir.create("../output", showWarnings = FALSE) # Crea la carpeta si no existe

# Creamos un nombre de archivo normalizado (ej: resumen_brasil.csv)
file_name = file.path("../output", paste0("resumen_", codigo_pais_residente))

# Exportación a CSV (Formato abierto)
write_csv(resumen, paste0(file_name, ".csv"))

# Exportación a RDS (Formato nativo de R: preserva factores y tipos de datos)
saveRDS(resumen, paste0(file_name, ".rds"))
}

```

```

[1] "Procesando: Brasil"
# A tibble: 4 x 4
  trimestre porcentaje_CO turistas_no_residentes_BA turistas_no_residentes_CO
  <fct>         <dbl>          <dbl>          <dbl>
1 1             1.65        1169674        19646
2 2             1.24        1286842        16186
3 3             1.56        1763087        27974
4 4             1.78        1400661        25412
[1] "Procesando: Chile"
# A tibble: 4 x 4
  trimestre porcentaje_CO turistas_no_residentes_BA turistas_no_residentes_CO
  <fct>         <dbl>          <dbl>          <dbl>
1 1             4.83        548407         27843
2 2             4.88        496884         25516
3 3             5.49        564384         32780
4 4             5.80        600177         36921
[1] "Procesando: No limítrofes"
# A tibble: 4 x 4
  trimestre porcentaje_CO turistas_no_residentes_BA turistas_no_residentes_CO

```

	<fct>	<dbl>	<dbl>	<dbl>
1	1	2.76	3995161	113447
2	2	3.02	2425453	75476
3	3	3.36	2347456	81620
4	4	2.88	3507778	103963

Veamos cómo podemos cargar los datos y organizarlos juntos de forma conveniente. La función `map_df` de la librería `purrr` (parte del `tidyverse`) automatiza la carga de múltiples archivos:

- **países:** es el vector con los nombres de los archivos a cargar (“brasil”, “chile”, “nolimitrofe”).
- **readRDS(...):** carga cada archivo en formato nativo de R desde la carpeta `../output`.
- **mutate(pais = .x):** agrega una columna nueva para identificar de qué archivo provienen los datos antes de unirlos.

Al terminar, `map_df` une automáticamente todas las tablas en un único dataframe consolidado.

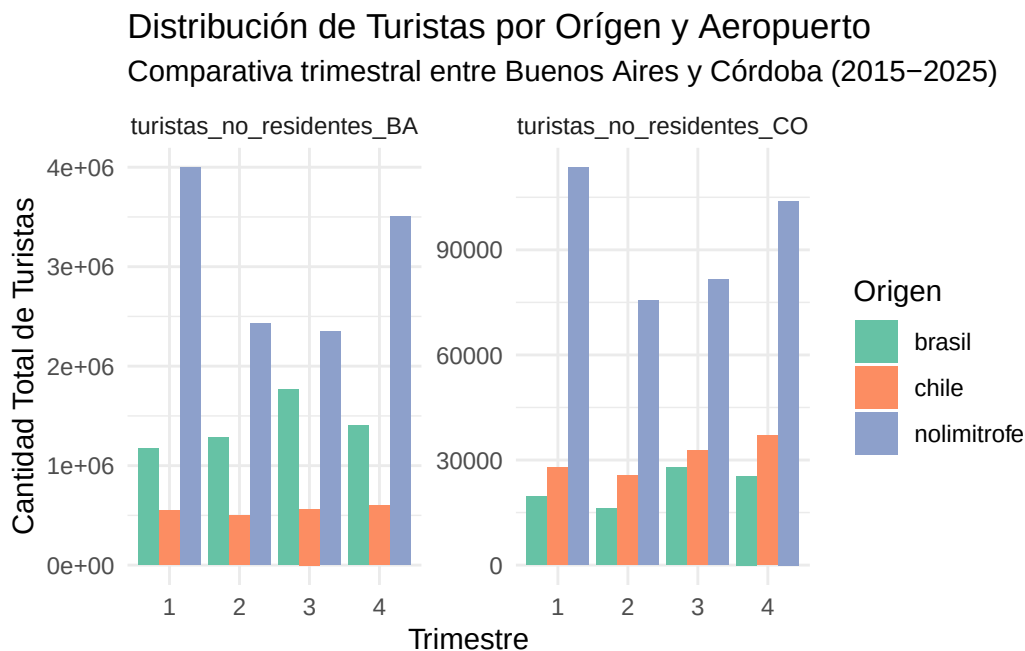
```
# Cargamos y unimos los datos de la carpeta output
países = c("brasil", "chile", "nolimitrofe")
datos_consolidados = map_df(
  países,
  ~ readRDS(file.path("../output", paste0("resumen_", .x, ".rds")))) |> mutate(pais = .x) #
)
```

Lo que sigue es necesario para hacer un gráfico, que no va a ser un requisito del TP, es sólo para tener una idea de lo que anduvimos haciendo.

```
# Transformamos a formato LONG para separar por Provincia
datos_histograma = datos_consolidados |>
  pivot_longer(
    cols = c(turistas_no_residentes_BA, turistas_no_residentes_CO), # qué columnas colapsar
    names_to = "provincia", # el nombre de la nueva columna que contendrá los antiguos nombres
    values_to = "total_turistas" # el nombre de la nueva columna que contendrá los valores n
  )
```

Notar que al utilizar las funciones de la librería `forcats` (`fct_recode` y `fct_reorder`) dentro del `mutate`, transformamos esas columnas de texto plano (`character`) a **factores** (`factor`), es decir, que le decimos a R que conserve un orden jerárquico acorde a un criterio.

```
library(ggplot2)
ggplot(datos_histograma, aes(x = trimestre, y = total_turistas, fill = pais)) +
  geom_bar(stat = "identity", position = "dodge") +
  facet_wrap(~ provincia, scales = "free_y") + # Crea un gráfico por provincia con su propia
  theme_minimal() +
  labs(
    title = "Distribución de Turistas por Origen y Aeropuerto",
    subtitle = "Comparativa trimestral entre Buenos Aires y Córdoba (2015-2025)",
    x = "Trimestre", # el orden garantizado por que es factors
    y = "Cantidad Total de Turistas",
    fill = "Origen"
  ) +
  scale_fill_brewer(palette = "Set2")
```



4. Correr scripts en segundo plano

Correr un script como **Background Job** en RStudio permite que R ejecute el código en una sesión separada, dejándote la consola libre para seguir analizando datos o probando funciones en la consola actual. Para correr un script de esta forma hay que guardar el archivo e ir a Background jobs -> Start Background Job. Seleccionamos el .R y el **directorio desde donde se corre**. De esta forma, si estamos

procesando un análisis exploratorio (como este), pero en simultaneo queremos correr un archivo que puede tomar mucho tiempo podemos ponerlo a correr en segundo plano.